

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

注意力机制即是一切所需

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

主流的序列转换模型基于复杂的循环或卷积神经网络，包含编码器和解码器。性能最优的模型还通过注意力机制连接编码器与解码器。我们提出一种全新的简单网络架构——Transformer，完全基于注意力机制，彻底摒弃了循环和卷积结构。在两个机器翻译任务上的实验表明，这些模型在质量上更优，同时具有更高的可并行性，所需训练时间显著减少。在 WMT 2014 英德翻译任务上，我们的模型取得了 28.4 BLEU 的成绩，比此前最佳结果（包括集成模型）提升了超过 2 BLEU。在 WMT 2014 英法翻译任务上，我们的模型在八块 GPU 上训练 3.5 天后，创下了单模型最新的 BLEU 分数 41.8，仅为文献中最佳模型训练成本的一小部分。我们通过英语句法成分解析任务上的成功应用——无论在大规模还是有限训练数据条件下——证明了 Transformer 具备良好的任务泛化能力。

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Łukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

1 引言

循环神经网络，特别是长短期记忆网络 [13] 和门控循环神经网络 [7]，已在序列建模和转换问题（如语言建模和机器翻译）中被牢固确立为最先进的方法 [35, 2, 5]。此后，众多研究不断推进循环语言模型和编码器-解码器架构的边界 [38, 24, 15]。

循环模型通常沿着输入和输出序列的符号位置分解计算。它们将位置与计算时间步对齐，生成隐藏状态序列 h_t ，作为前一隐藏状态 h_{t-1} 和位置 t 的输入的函数。这种固有的顺序性阻碍了训练样本内部的并行化，在序列长度较长时尤为关键，因为内存限制制约了样本间的批处理。近期研究通过分解技巧 [21] 和条件计算 [32] 在计算效率方面取得了显著改进，后者还同时提升了模型性能。然而，顺序计算的根约束依然存在。

注意力机制已成为各种任务中卓越序列建模和转换模型的核心组成部分，允许在不考虑输入或输出序列中距离的情况下建模依赖关系 [2, 19]。然而，除少数例外 [27]，这类注意力机制通常与循环网络联合使用。

本文提出 Transformer，一种摒弃循环、完全依赖注意力机制来建立输入与输出之间全局依赖关系的模型架构。Transformer 支持更大程度的并行化，在八块 P100 GPU 上仅训练十二小时即可达到翻译质量的最新水平。

2 背景

降低顺序计算量的目标也是 Extended Neural GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础。它们均以卷积神经网络为基本构建块，对所有输入输出位置并行计算隐藏表示。在这些模型中，关联两个任意输入或输出位置信号所需的操作数随位置间距离增长——ConvS2S 为线性增长，ByteNet 为对数增长。这使得学习远距离位置间的依赖关系更为困难 [12]。在 Transformer 中，这一操作数被缩减为常数，尽管代价是因注意力加权位置的平均化而导致有效分辨率降低，我们通过第 3.2 节描述的多头注意力来抵消这一影响。

自注意力（有时称为内部注意力）是一种关联单个序列中不同位置以计算序列表示的注意力机制。自注意力已成功应用于多种任务，包括阅读理解、抽象摘要、文本蕴含以及学习任务无关的句子表示 [4, 27, 28, 22]。

端到端记忆网络基于循环注意力机制而非序列对齐的循环结构，并在简单语言问答和语言建模任务上表现良好 [34]。

然而，据我们所知，Transformer 是第一个完全依赖自注意力来计算输入和输出表示、而不使用序列对齐 RNN 或卷积的转换模型。在后续章节中，我们将详细描述 Transformer，阐述自注意力的动机，并讨论其相较于 [17, 18] 和 [9] 等模型的优势。

3 模型架构

大多数有竞争力的神经序列转换模型采用编码器-解码器结构 [5, 2, 35]。其中，编码器将输入符号表示序列 $(x_1, \dots, x_n)$ 映射为连续表示序列 $\mathbf{z} = (z_1, \dots, z_n)$ 。给定 $\mathbf{z}$ ，解码器随后一次生成一个输出符号序列 $(y_1, \dots, y_m)$ 。在每一步，模型以自回归方式运行 [10]，在生成下一个符号时将已生成的符号作为额外输入。

Transformer 遵循这一整体架构，编码器和解码器均使用堆叠的自注意力和逐位置的全连接层，分别如图 1 左右两半所示。

3.1 Encoder and Decoder Stacks

编码器： 编码器由 $N = 6$ 个相同层堆叠而成。每层包含两个子层。第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在两个子层的周围分别引入残差连接 [11]，随后进行层归一化 [1]。即每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 为该子层自身实现的函数。为便于残差连接，模型中所有子层以及嵌入层均产生维度 $d_{\text{model}} = 512$ 的输出。

解码器： 解码器同样由 $N = 6$ 个相同层堆叠而成。除了每个编码器层中的两个子层外，解码器插入了第三个子层，该子层对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个

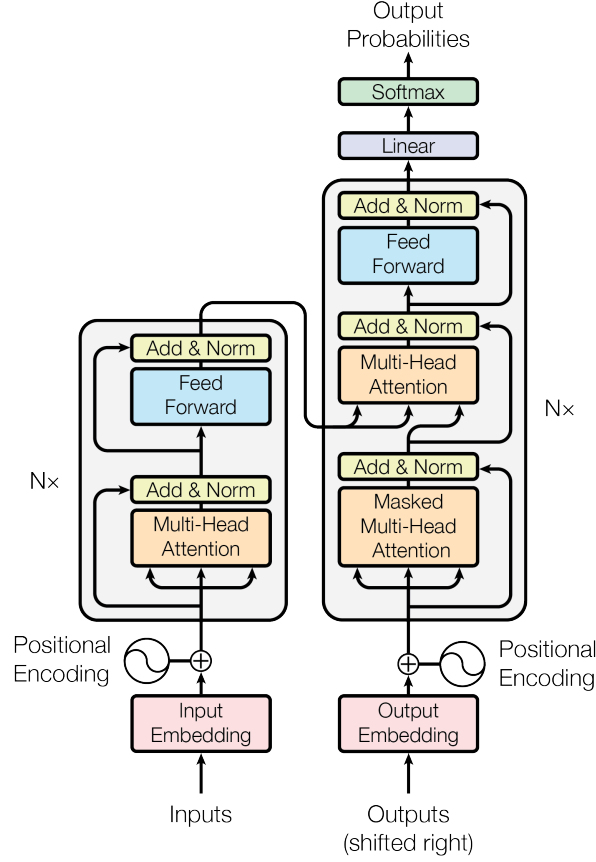


Figure 1: Transformer 模型架构。

子层周围引入残差连接，随后进行层归一化。我们还对解码器堆栈中的自注意力子层进行了修改，以防止位置注意到后续位置。这种掩码操作，加上输出嵌入偏移一个位置的事实，确保位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2 Attention

注意力函数可以描述为将查询和一组键值对映射到输出，其中查询、键、值和输出均为向量。输出由值的加权和计算得出，分配给每个值的权重由查询与相应键的兼容性函数计算得到。

3.2.1 Scaled Dot-Product Attention

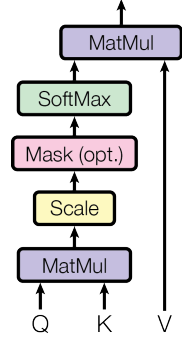
我们将所采用的注意力称为“缩放点积注意力”（图 2）。输入由维度为 d_k 的查询和键，以及维度为 d_v 的值组成。我们计算查询与所有键的点积，各除以 $\sqrt{d_k}$ ，再应用 softmax 函数以获得值上的权重。

实践中，我们同时对一组查询计算注意力函数，将其打包成矩阵 Q 。键和值也分别打包成矩阵 K 和 V 。输出矩阵的计算公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

两种最常用的注意力函数是加性注意力 [2] 和点积（乘性）注意力。点积注意力与我们的算法相同，只是缺少了缩放因子 $\frac{1}{\sqrt{d_k}}$ 。加性注意力使用单隐层前馈网络计算兼容性函数。虽然两者在理论复杂度上相似，但点积注意力在实践中速度更快且更节省空间，因为它可以利用高度优化的矩阵乘法代码实现。

Scaled Dot-Product Attention



Multi-Head Attention

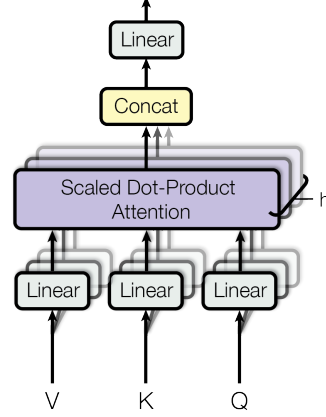


Figure 2: (左) 缩放点积注意力。(右) 多头注意力由多个并行运行的注意力层组成。

尽管在 d_k 值较小时两种机制表现相近，但当 d_k 较大时，加性注意力优于无缩放的点积注意力 [3]。我们推测，对于较大的 d_k ，点积的幅度增大，将 softmax 函数推至极小梯度区域⁴。为抵消这一效应，我们将点积缩放 $\frac{1}{\sqrt{d_k}}$ 。

3.2.2 Multi-Head Attention

我们发现，与其使用 d_{model} 维的键、值和查询执行单个注意力函数，不如将查询、键和值分别用不同的可学习线性投影进行 h 次投影，映射到 d_k 、 d_k 和 d_v 维度，效果更佳。随后，我们对这些投影后的查询、键和值并行执行注意力函数，产生 d_v 维的输出值。这些输出被拼接并再次投影，得到最终值，如图 2 所示。

多头注意力允许模型在不同位置上共同关注来自不同表示子空间的信息。单头注意力中，平均化会抑制这种能力。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$.

本文中我们使用 $h = 8$ 个并行注意力层（或称为头）。每个头使用 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度降低，总计算成本与全维度的单头注意力相近。

3.2.3 Applications of Attention in our Model

Transformer 以三种不同方式使用多头注意力：

- 在“编码器-解码器注意力”层中，查询来自前一解码器层，记忆键和值来自编码器的输出。这使得解码器中的每个位置都能关注输入序列中的所有位置。这模仿了序列到序列模型中典型的编码器-解码器注意力机制，如 [38, 2, 9]。
- 编码器包含自注意力层。在自注意力层中，所有键、值和查询均来自同一处——即编码器中前一层的输出。编码器中每个位置都可关注编码器前一层中的所有位置。

⁴为说明点积为何变大，假设 q 和 k 的分量是均值为 0、方差为 1 的独立随机变量。则其点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为 0、方差为 d_k 。

- 类似地，解码器中的自注意力层允许解码器中每个位置关注包括自身在内的所有解码器位置。我们需要阻止解码器中向左的信息流以保持自回归性质。我们通过在缩放点积注意力中对 softmax 输入中所有对应非法连接的值进行掩码（设为 $-\infty$ ）来实现这一点。参见图 2。

3.3 Position-wise Feed-Forward Networks

除注意力子层外，编码器和解码器中的每一层还包含一个全连接前馈网络，该网络对每个位置分别且相同地应用。它由两个线性变换组成，中间夹有 ReLU 激活函数。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然线性变换在不同位置上是相同的，但它们在不同层之间使用不同的参数。另一种描述方式是两个核大小为 1 的卷积。输入和输出的维度为 $d_{\text{model}} = 512$ ，内层维度为 $d_{\text{ff}} = 2048$ 。

3.4 Embeddings and Softmax

与其他序列转换模型类似，我们使用可学习的嵌入将输入和输出 Token 转换为维度 d_{model} 的向量。我们也使用常规的可学习线性变换和 softmax 函数将解码器输出转换为预测的下一个 Token 概率。在我们的模型中，两个嵌入层和 softmax 前的线性变换共享同一权重矩阵，类似于 [30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$ 。

3.5 Positional Encoding

由于我们的模型不包含循环和卷积，为使模型能够利用序列的顺序信息，我们必须注入一些关于序列中 Token 相对或绝对位置的信息。为此，我们在编码器和解码器堆栈底部的输入嵌入上添加“位置编码”。位置编码与嵌入具有相同的维度 d_{model} ，因此两者可以相加。位置编码有多种选择，包括可学习的和固定的 [9]。

本文中，我们使用不同频率的正弦和余弦函数：

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{\text{model}}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{\text{model}}}) \end{aligned}$$

其中 pos 为位置， i 为维度。即位置编码的每个维度对应一个正弦波。波长形成从 2π 到 $10000 \cdot 2\pi$ 的几何级数。我们选择这一函数是因为我们假设它能让模型轻松学会按相对位置进行关注，因为对于任意固定偏移 k ， PE_{pos+k} 可表示为 PE_{pos} 的线性函数。

我们也尝试使用可学习的位置嵌入 [9] 作为替代方案，发现两种版本产生了几乎相同的结果（见表 3 行 (E)）。我们选择正弦版本，因为它可能允许模型外推到训练期间未见过的更长序列长度。

4 为什么选择自注意力

本节，我们将自注意力层与循环层和卷积层进行多方面的比较。这些层通常用于将一个变长符号表示序列 $(x_1, \dots, x_n)$ 映射到另一个等长序列 $(z_1, \dots, z_n)$ ，其中 $x_i, z_i \in \mathbb{R}^d$ ，例如典型序列转换编码器或解码器中的隐藏层。为阐述使用自注意力的动机，我们考虑了三个需求指标。

其一是每层的总计算复杂度。其二是可并行化的计算量，以所需的最少顺序操作数衡量。

其三为网络中长程依赖之间的路径长度。学习长程依赖是许多序列转换任务中的关键挑战。影响学习此类依赖能力的一个关键因素是前向和反向信号在网络中传播路径的长度。输入输出序列中任意位置组合之间的路径越短，学习长程依赖就越容易 [12]。因此，我们还比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表 1 所示，自注意力层以常数数量的顺序执行操作连接所有位置，而循环层需要 $O(n)$ 次顺序操作。在计算复杂度方面，当序列长度 n 小于表示维度 d 时，自注意力层比循环层更快，这在机器翻译中最先进的句子表示（如 word-piece [38] 和 byte-pair [31] 表示）中最为常见。为提升处理超长序列任务的计算性能，可以将自注意力限制为仅考虑以各输出位置为中心的输入

Table 1: 不同层类型的最大路径长度、每层复杂度及最少顺序操作数。 n 为序列长度, d 为表示维度, k 为卷积核大小, r 为受限自注意力中邻域的大小。

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

序列中大小为 r 的邻域。这会将最大路径长度增至 $O(n/r)$ 。我们计划在未来的工作中进一步研究这一方案。

单个核宽度为 $k < n$ 的卷积层无法连接所有输入输出位置对。要实现全覆盖, 需堆叠 $O(n/k)$ 个连续卷积层 (连续核情况), 或 $O(\log_k(n))$ 个空洞卷积层 [18], 从而增加网络中任意两位置间的最长路径。卷积层通常比循环层昂贵 k 倍。然而, 可分离卷积 [6] 将复杂度显著降低至 $O(k \cdot n \cdot d + n \cdot d^2)$ 。即使 $k = n$, 可分离卷积的复杂度也等同于自注意力层与逐位置前馈层的组合——这正是我们模型采用的方法。

作为附带好处, 自注意力可能产生更具可解释性的模型。我们从模型中检查注意力分布, 并在附录中展示和讨论相关示例。不仅各个注意力头明确学会了执行不同任务, 许多头部还表现出与句子句法和语义结构相关的行为。

5 训练

本节描述模型的训练方案。

5.1 Training Data and Batching

我们在标准 WMT 2014 英德数据集上训练, 该数据集包含约 450 万句子对。句子使用 byte-pair 编码 [3], 共享源-目标词汇量约 37000 个 Token。对于英法翻译, 我们使用规模显著更大的 WMT 2014 英法数据集 (3600 万句子), 并将 Token 分割为 32000 个 word-piece 词汇 [38]。句子对按近似序列长度分批。每个训练批次包含一组句子对, 约含 25000 个源 Token 和 25000 个目标 Token。

5.2 Hardware and Schedule

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练模型。对于使用全文中描述的超参数的基础模型, 每个训练步骤耗时约 0.4 秒。基础模型总共训练 100,000 步, 即 12 小时。对于大模型 (见表 3 底行), 每步耗时 1.0 秒。大模型训练了 300,000 步 (3.5 天)。

5.3 Optimizer

We used the Adam optimizer [20] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

这对应于在前 $warmup_steps$ 训练步中线性增加学习率, 随后按步数反平方根的比例递减。我们取 $warmup_steps = 4000$ 。

5.4 Regularization

训练期间我们采用三种正则化方式:

Table 2: Transformer 在英德和英法 newstest2014 测试集上以远低于前人训练成本取得了比以往最先进模型更好的 BLEU 分数。

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

残差 Dropout 我们在每个子层输出被加到子层输入并进行归一化之前，对其应用 Dropout [33]。此外，我们在编码器和解码器堆栈中的嵌入与位置编码之和上也应用 Dropout。基础模型使用比率 $P_{drop} = 0.1$ 。

标签平滑 训练期间，我们使用值为 $\epsilon_{ls} = 0.1$ 的标签平滑 [36]。这会损害困惑度，因为模型学会了更加不确定，但能提高准确率和 BLEU 分数。

6 实验结果

6.1 Machine Translation

在 WMT 2014 英德翻译任务上，大 Transformer 模型（表 2 中的 Transformer (big)）比此前最佳模型（包括集成模型）高出 2.0 BLEU 以上，创下了 28.4 的最新 BLEU 分数。该模型的配置列于表 3 最末行。训练在 8 块 P100 GPU 上耗时 3.5 天。即使是我们的基础模型也超越了此前所有已发表的模型和集成模型，且训练成本仅为任一竞争模型的零头。

在 WMT 2014 英法翻译任务上，我们的大模型达到了 41.0 BLEU，超越了此前所有已发表的单模型，且训练成本不到此前最佳模型的 1/4。用于英法翻译的 Transformer (big) 模型使用 Dropout 率 $P_{drop} = 0.1$ ，而非 0.3。

对于基础模型，我们使用由最后 5 个检查点平均得到的单一模型，这些检查点每 10 分钟保存一次。对于大模型，我们对最后 20 个检查点进行平均。我们使用束大小为 4、长度惩罚 $\alpha = 0.6$ 的束搜索 [38]。这些超参数是在开发集上实验后选定的。我们在推理时将最大输出长度设为输入长度 +50，但在可能时提前终止 [38]。

表 2 总结了我们的结果，并将翻译质量和训练成本与文献中其他模型架构进行了比较。我们通过将训练时间、使用的 GPU 数量以及每个 GPU 的持续单精度浮点性能估算值相乘来估算训练模型所用的浮点运算数⁵。

6.2 Model Variations

为评估 Transformer 各组件的重要性，我们对基础模型进行了多方面的变体实验，测量英德翻译在开发集 newstest2013 上的性能变化。我们使用前述的束搜索，但未使用检查点平均。结果列于表 3。

在表 3 行 (A) 中，我们在保持计算量不变的前提下改变注意力头数量以及注意力键和值的维度（如第 3.2.2 节所述）。单头注意力比最佳设置低 0.9 BLEU，但注意力头过多也会导致质量下降。

在表 3 行 (B) 中，我们观察到减小注意力键尺寸 d_k 会损害模型质量。这表明确定兼容性并非易事，比点积更复杂的兼容性函数可能有好处。我们进一步在行 (C) 和 (D) 中观察到，如预期

⁵我们分别对 K80、K40、M40 和 P100 使用了 2.8、3.7、6.0 和 9.5 TFLOPS 的值。

Table 3: Transformer 架构的变体实验。未列出的值与基础模型相同。所有指标基于英德翻译开发集 newstest2013。报告的困惑度为按 byte-pair 编码的 per-wordpiece，不应与 per-word 困惑度直接比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$	
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65	
(A)					1	512	512			5.29	24.9		
					4	128	128			5.00	25.5		
					16	32	32			4.91	25.8		
					32	16	16			5.01	25.4		
(B)					16					5.16	25.1	58	
					32					5.01	25.4	60	
(C)	2									6.11	23.7	36	
	4									5.19	25.3	50	
	8									4.88	25.5	80	
	256				32	32				5.75	24.5	28	
	1024				128	128				4.66	26.0	168	
			1024							5.12	25.4	53	
			4096							4.75	26.2	90	
(D)					0.0				5.77	24.6			
					0.2				4.95	25.5			
					0.0				4.67	25.3			
					0.2				5.47	25.7			
(E)	positional embedding instead of sinusoids									4.92	25.7		
big	6	1024	4096	16	0.3				300K	4.33	26.4	213	

Table 4: Transformer 在英语句法成分解析任务上表现良好（结果为 WSJ 第 23 节数据）

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

一样，更大的模型更好，且 Dropout 非常有助于防止过拟合。在行 (E) 中，我们将正弦位置编码替换为可学习的位置嵌入 [9]，观察到与基础模型几乎相同的结果。

6.3 English Constituency Parsing

为评估 Transformer 能否泛化到其他任务，我们在英语句法成分解析上进行了实验。该任务具有特殊挑战：输出受强结构约束且显著长于输入。此外，RNN 序列到序列模型在小数据场景下一直未能取得最先进结果 [37]。

我们在 Penn Treebank [25] 的 Wall Street Journal (WSJ) 部分上训练了一个 4 层 Transformer， $d_{\text{model}} = 1024$ ，约 40K 训练句。我们还在半监督设置下进行训练，使用了更大的高置信度语料和 BerkeleyParser 语料，总计约 1700 万句子 [37]。WSJ 单独设置使用 16K Token 词汇表，半监督设置使用 32K Token 词汇表。

我们仅在 Section 22 开发集上进行了少量实验以选择 Dropout (注意力 Dropout 和残差 Dropout, 见第 5.4 节)、学习率和束大小, 其他所有参数均与英德基础翻译模型保持一致。推理期间, 我们将最大输出长度增加到输入长度 +300。WSJ 单独和半监督设置均使用束大小为 21、 $\alpha = 0.3$ 。

表 4 的结果表明, 尽管缺乏特定于任务的调优, 我们的模型表现惊人地出色, 结果优于除 Recurrent Neural Network Grammar [8] 外的所有此前报道的模型。

与 RNN 序列到序列模型 [37] 不同, 即使在仅包含 40K 句子的 WSJ 训练集上训练, Transformer 也超越了 BerkeleyParser [29]。

7 结论

本文提出了 Transformer, 这是第一个完全基于注意力机制的序列转换模型, 用多头自注意力取代了编码器-解码器架构中最常用的循环层。

在翻译任务上, Transformer 的训练速度相比基于循环或卷积层的架构有显著提升。在 WMT 2014 英德和 WMT 2014 英法两个翻译任务上, 我们都取得了最新的最佳成绩。在前者上, 我们最好的模型甚至超越了此前所有报道过的集成模型。

我们对基于注意力的模型的未来充满期待, 并计划将其应用于更多任务。我们计划将 Transformer 扩展到文本以外的输入输出模态问题, 并研究局部的、受限的注意力机制, 以高效处理图像、音频和视频等大型输入输出。使生成过程减少顺序依赖是我们的另一个研究目标。

训练和评估模型所使用的代码可在 <https://github.com/tensorflow/tensor2tensor> 获取。

致谢 我们感谢 Nal Kalchbrenner 和 Stephan Gouws 富有建设性的评论、修正与启发。

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.
- [5] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Łukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.
- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

Attention Visualizations

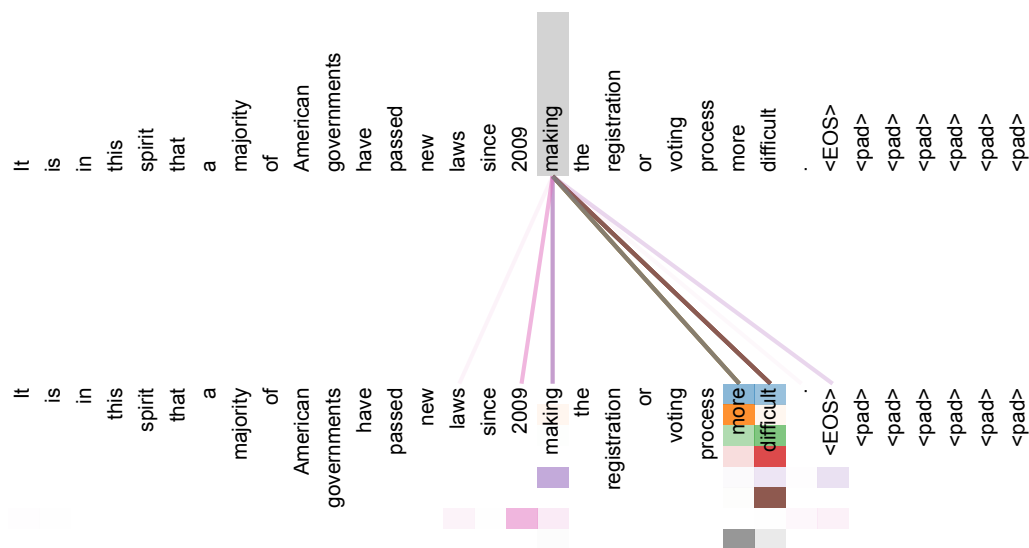


Figure 3: 注意力机制跟踪长程依赖的一个示例，来自 6 层中第 5 层编码器自注意力。多个注意力头关注动词 ‘making’ 的远距离依赖，补全了短语 ‘making...more difficult’。此处仅展示词 ‘making’ 的注意力分布。不同颜色代表不同注意力头。建议彩色查看。

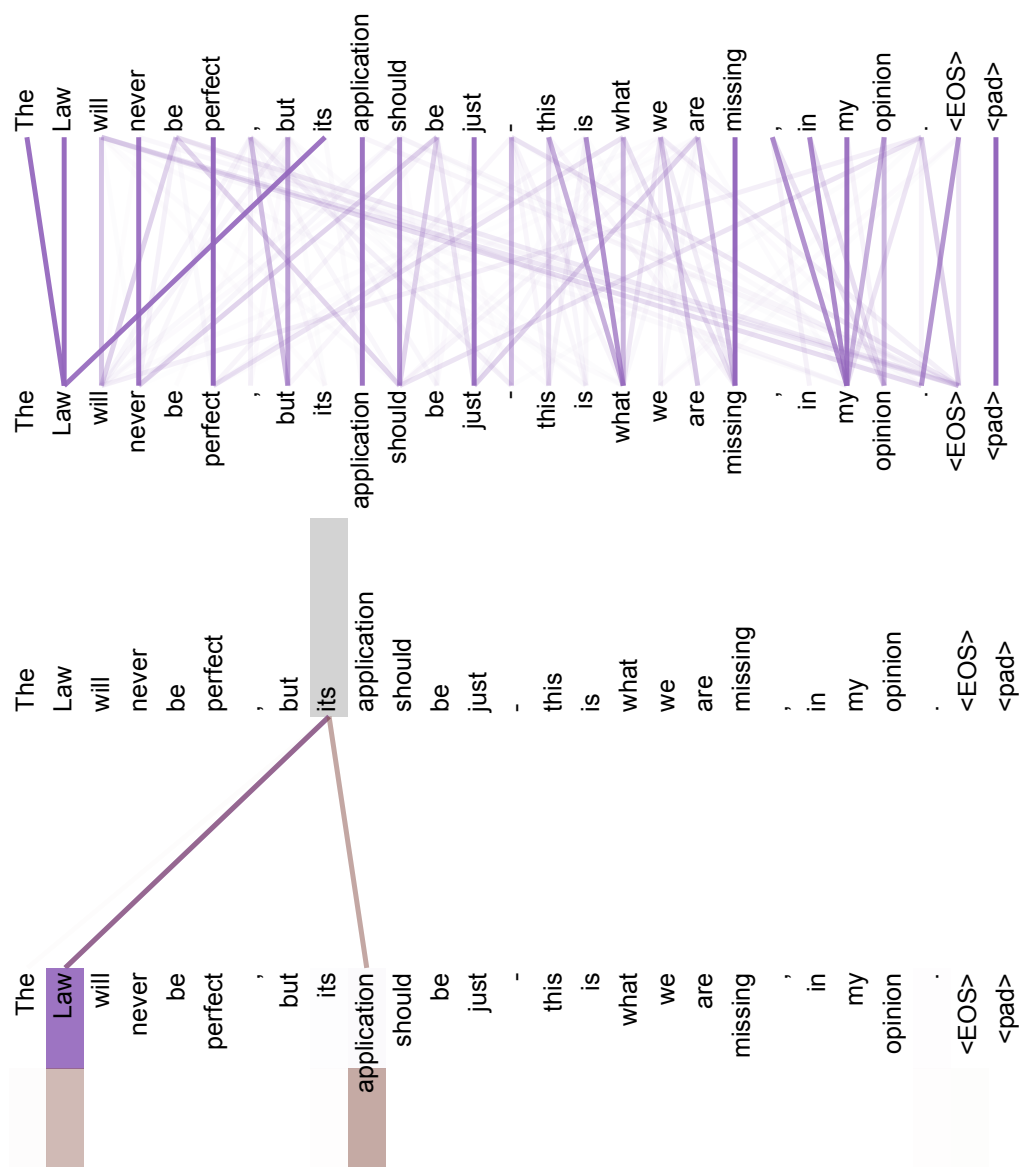


Figure 4: 同样来自 6 层中第 5 层的两个注意力头，显然参与了指代消解。上：注意力头 5 的完整注意力分布。下：仅限词 'its' 的注意力头 5 和 6 的独立注意力分布。注意该词的注意力非常尖锐。

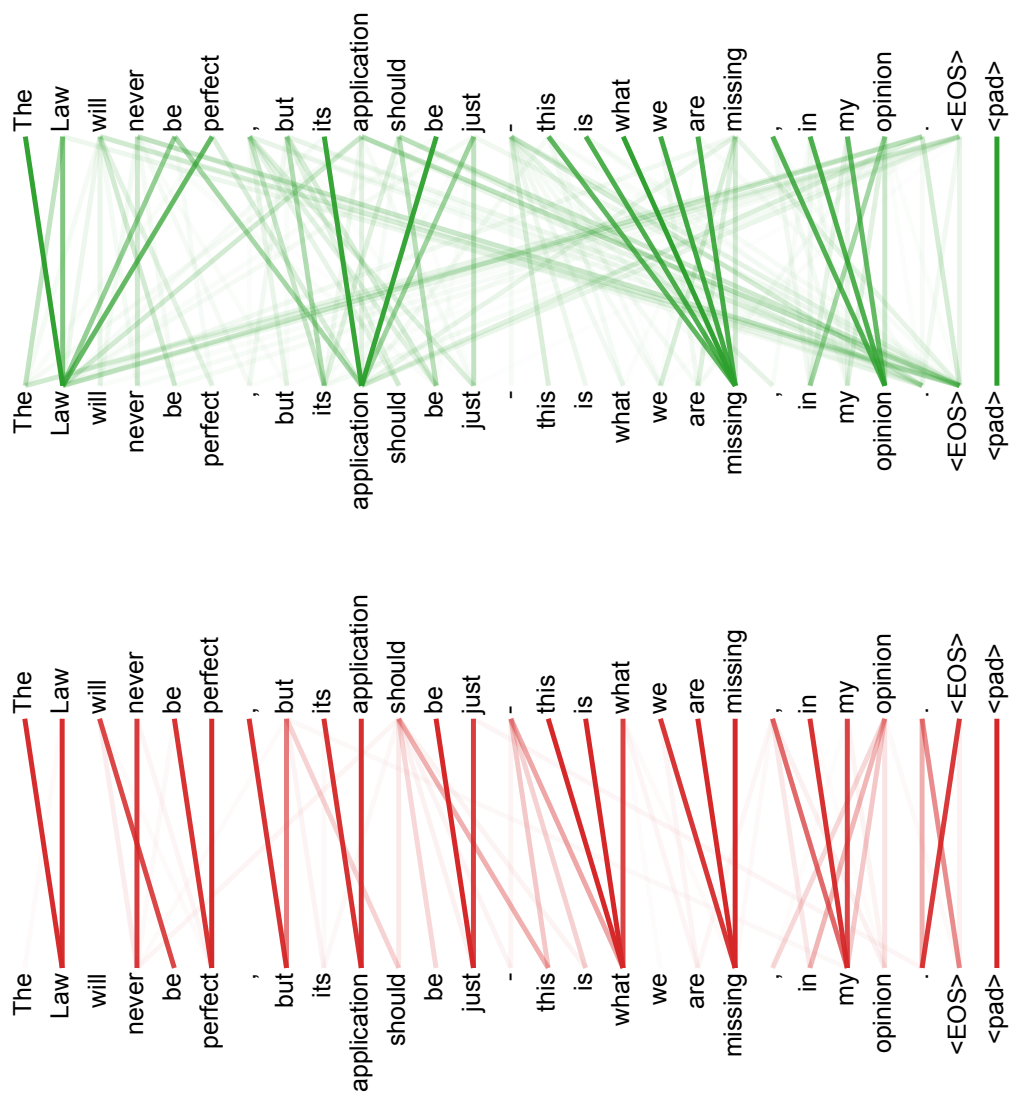


Figure 5: 许多注意力头展现出与句子结构相关的行为。以上给出两个示例，来自 6 层中第 5 层编码器自注意力的两个不同注意力头。这些头明确学会了执行不同的任务。